

高级搜索树

B-树：大数据

越来越小的内存

❖ RAM: 存储器? 不就是无限可数个寄存器吗?

Turing: 存储器? 不就是无限长的纸带吗?

❖ 但事实上

系统存储容量的增长速度

<< 应用问题规模的增长速度

1 Kilobyte = $2^{10} = 10^3$

1 Megabyte = $2^{20} = 10^6$

1 Gigabyte = $2^{30} = 10^9$

1 Terabyte = $2^{40} = 10^{12}$

1 Petabyte = $2^{50} = 10^{15}$

1 Exabyte = $2^{60} = 10^{18}$

2010 1 Zettabyte = $2^{70} = 10^{21}$

1 Yottabyte = $2^{80} = 10^{24}$

1 Nonabyte = $2^{90} = 10^{27}$

1 Doggabyte = $2^{100} = 10^{30}$

越来越小的内存

❖ 典型的 $1980: 10\text{MB} / 1\text{MB} = 10$

数据库规模/内存容量 2010: 1TB / 1GB = 1000

❖ 今天典型的数据集以TB度量；相对而言，内存容量在不断减小！

❖ 内存为何不能更大？存储器容量越大/小，访问速度越慢/快

❖ 事实1: 不同容量的存储器, 访问速度差异悬殊

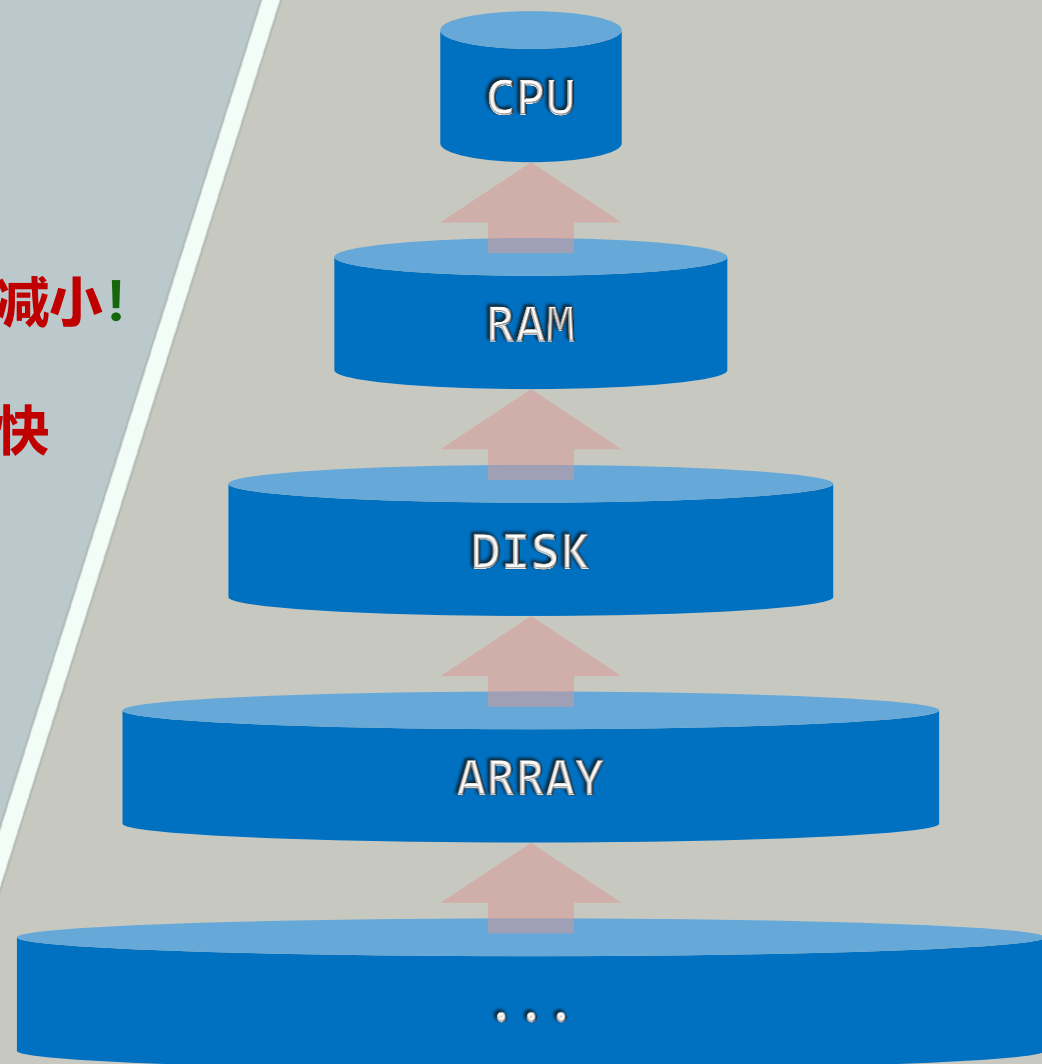
以磁盘与内存为例: $\text{ms/ns} > 10^5$

$$\frac{10^2}{10^1}$$

❖ 若一次内存访问需要一秒，则一次磁盘访问就需一天

为避免一次磁盘访问，我们宁愿访问内存1000次

❖ 实用的存储系统，多为分级结构...



分级存储

❖ 机制与策略

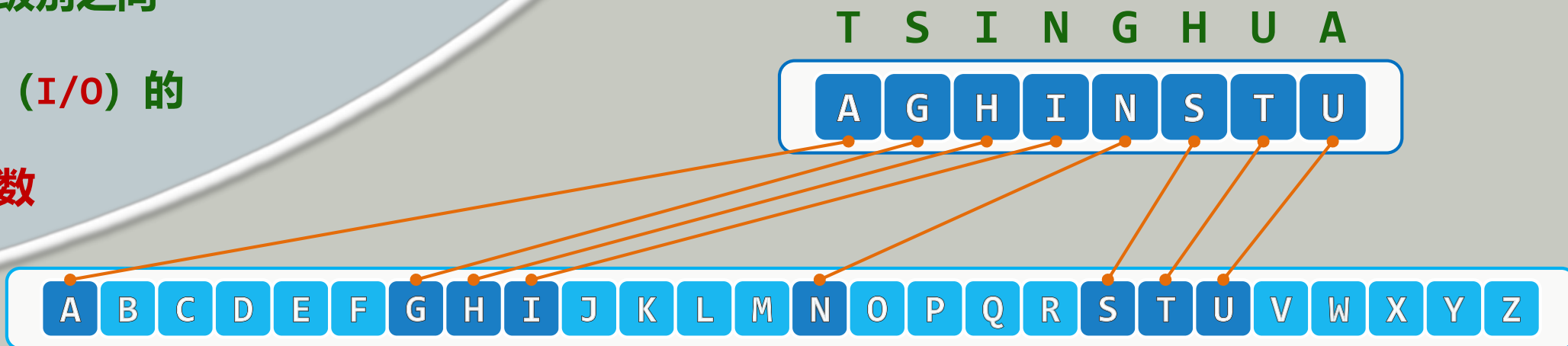
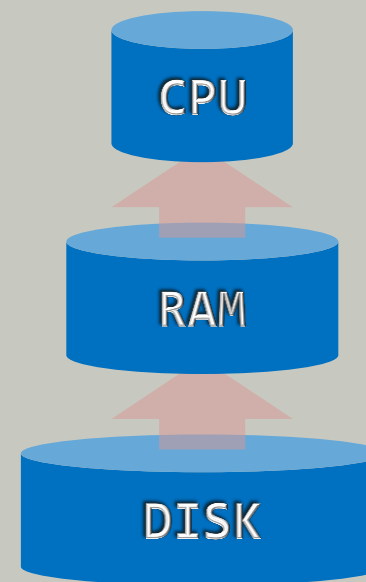
- 常用的数据，复制到更高层、更小的存储器中
- 找不到，才向更低层、更大的存储器索取

❖ 算法的实际运行时间，主要取决于

相邻存储级别之间

数据传输 (I/O) 的

速度与次数



缓存：从磁盘中读写1B，与读写1KB几乎一样快

❖ 批量式访问：以页（page）或块（block）为单位，使用缓冲区

```
// <stdio.h>
```

```
#define BUFSIZ 512 //缓冲区默认容量
```

```
int setvbuf( //定制缓冲区
```

```
FILE* fp, //流
```

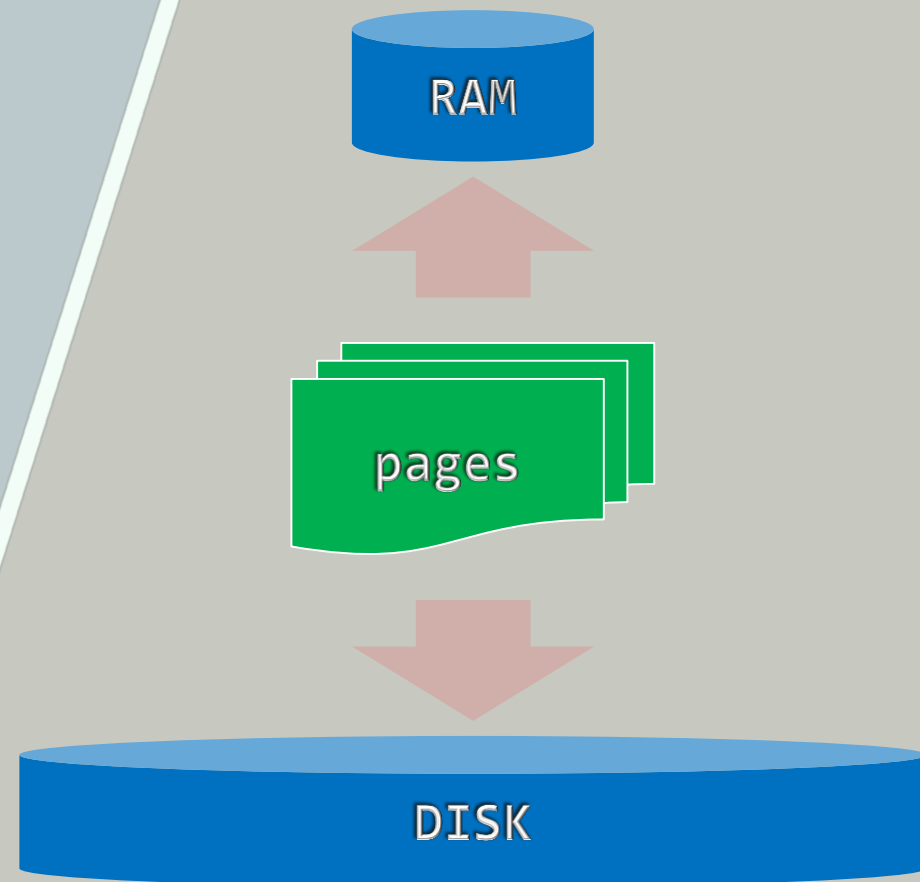
```
char* buf, //缓冲区
```

```
int _Mode, //_IOFBF | _IOLBF | _IONBF
```

```
size_t size); //缓冲区容量
```

```
int fflush( FILE* fp ); //强制清空缓冲区
```

❖ 效果：单位字节的平均访问时间大大缩短



高级搜索树

B-树： 结构

等价变换

❖ 平衡的多路搜索树

R. Bayer & E. McCreight

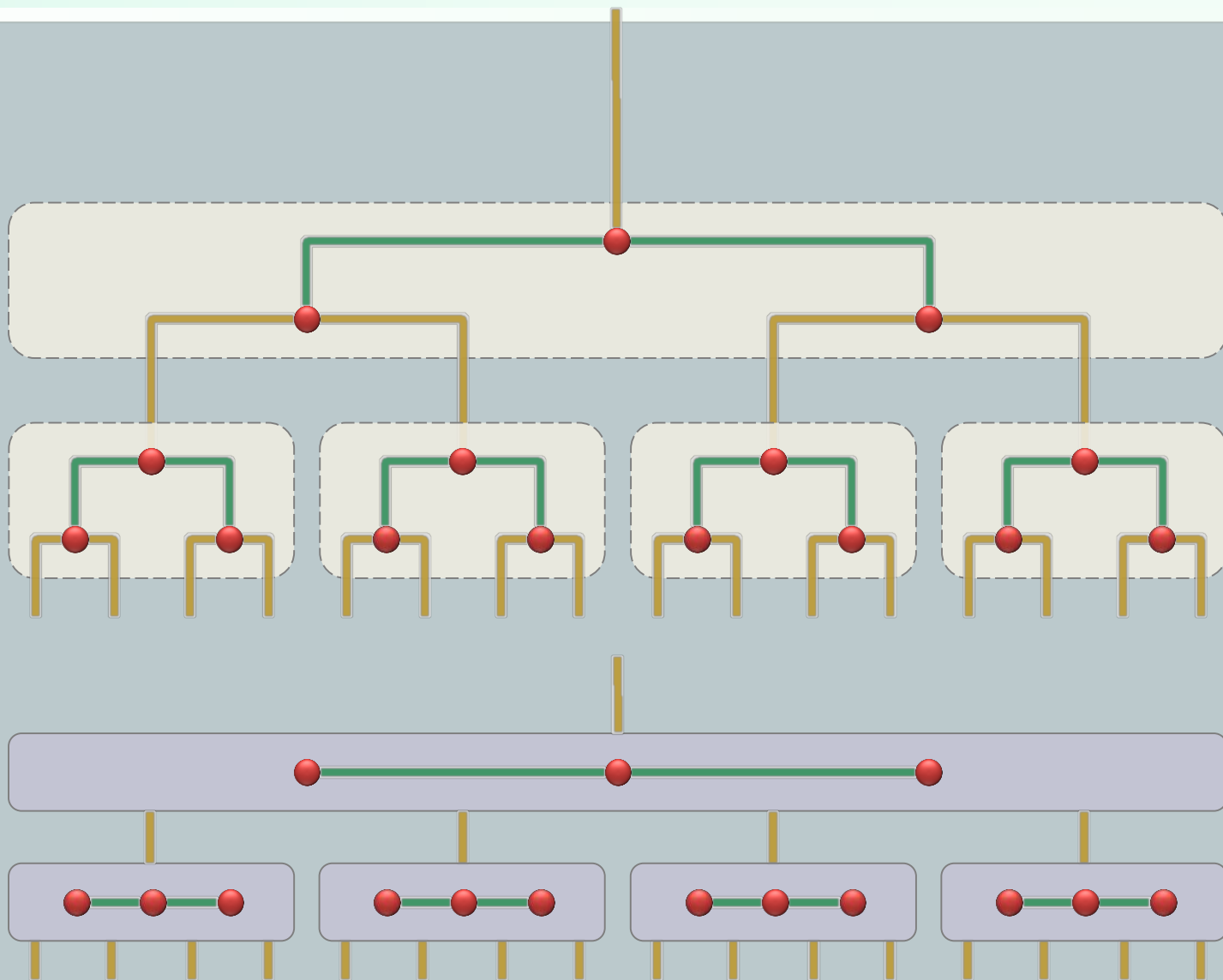
1970

❖ 每 d 代合并为超级节点

- $m = 2^d$ 路
- $m-1$ 个关键码

❖ 逻辑上与BBST完全等价

既如此，B-树之意义何在？



I/O优化: 多级存储系统中使用B-树, 可针对外部查找, 大大减少I/O次数

❖ 难道, AVL还不够? 比如, 若有 $n = 1G$ 个记录...

- 每次查找需要 $\log_2 10^9 \approx 30$ 次I/O操作
- 每次只读出单个关键码, 得不偿失

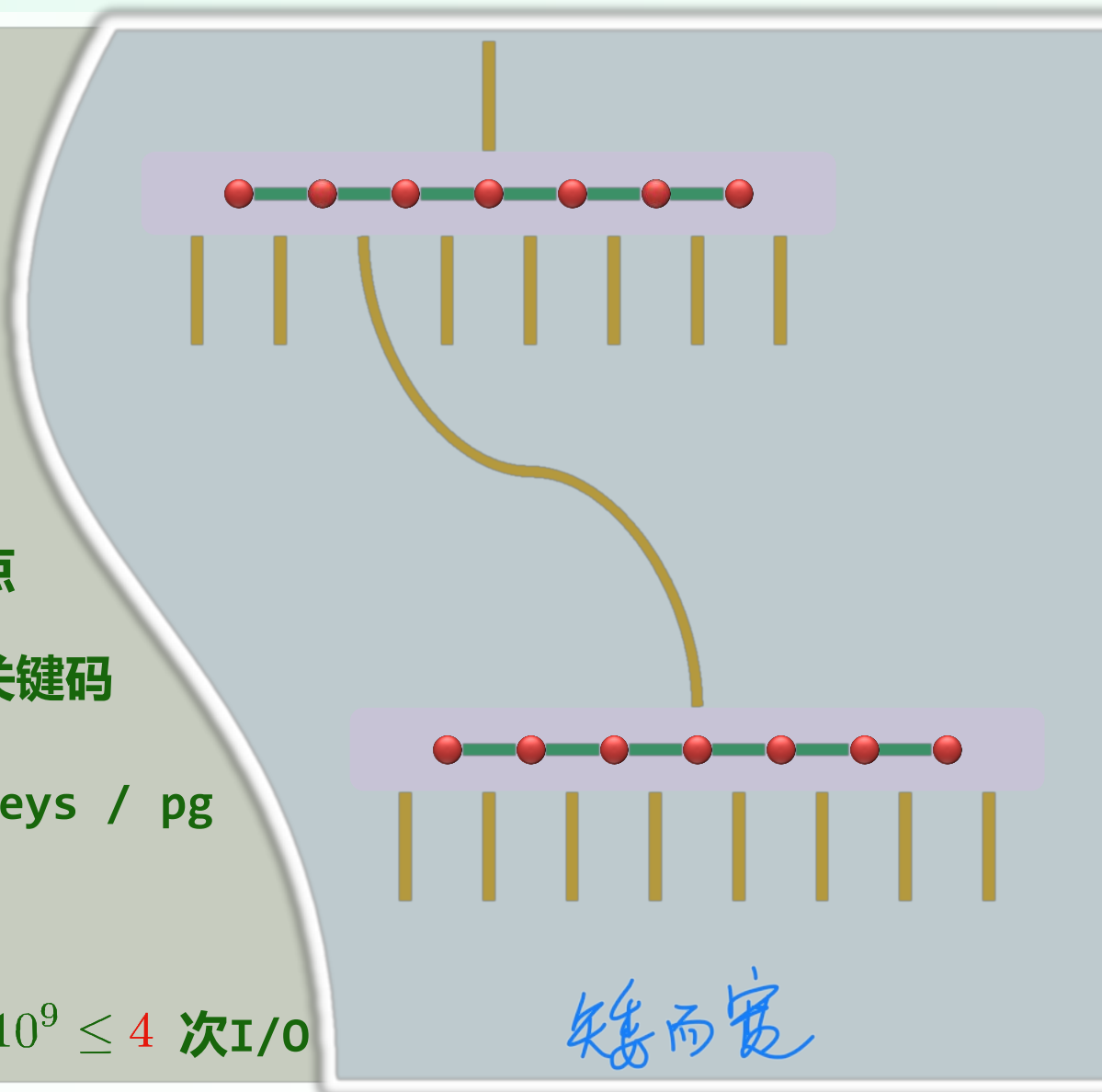
❖ B-树又能如何?

- 充分利用外存的批量访问, 将此特点转化为优点
- 每下降一层, 都以超级节点为单位, 读入一组关键码

❖ 具体多大一组? 视磁盘的数据块大小而定, $m = \#keys / pg$

- 比如, 目前多数数据库系统采用 $m = 200 \sim 300$

❖ 回到上例, 若取 $m = 256$, 则每次查找只需 $\log_{256} 10^9 \leq 4$ 次I/O



外部节点 + 叶子

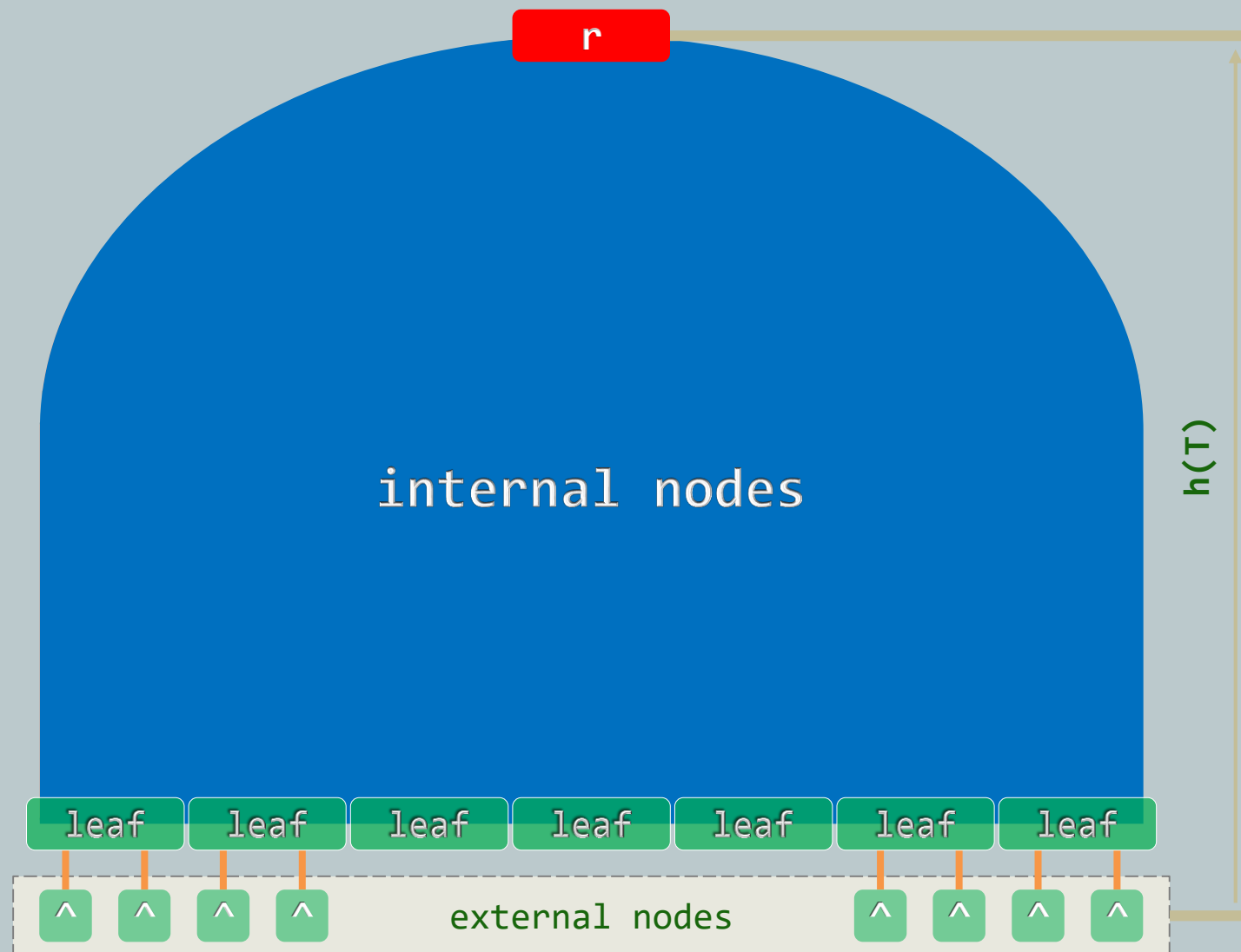
❖ 所谓**m阶B-树**，即

m路完全平衡搜索树 ($m \geq 3$)

❖ **外部节点**的深度统一相等
→ 叶子节点下的 (空节点)

约定以此深度作为树高 h

❖ **叶节点**的深度统一相等 ($h-1$)



内部节点

❖ 各含 $n \leq m-1$ 个关键码:

$$K_1 < K_2 < K_3 < \dots < K_n$$

❖ 各有 $n+1 \leq m$ 个分支:

$$A_0, A_1, A_2, A_3, \dots, A_n$$

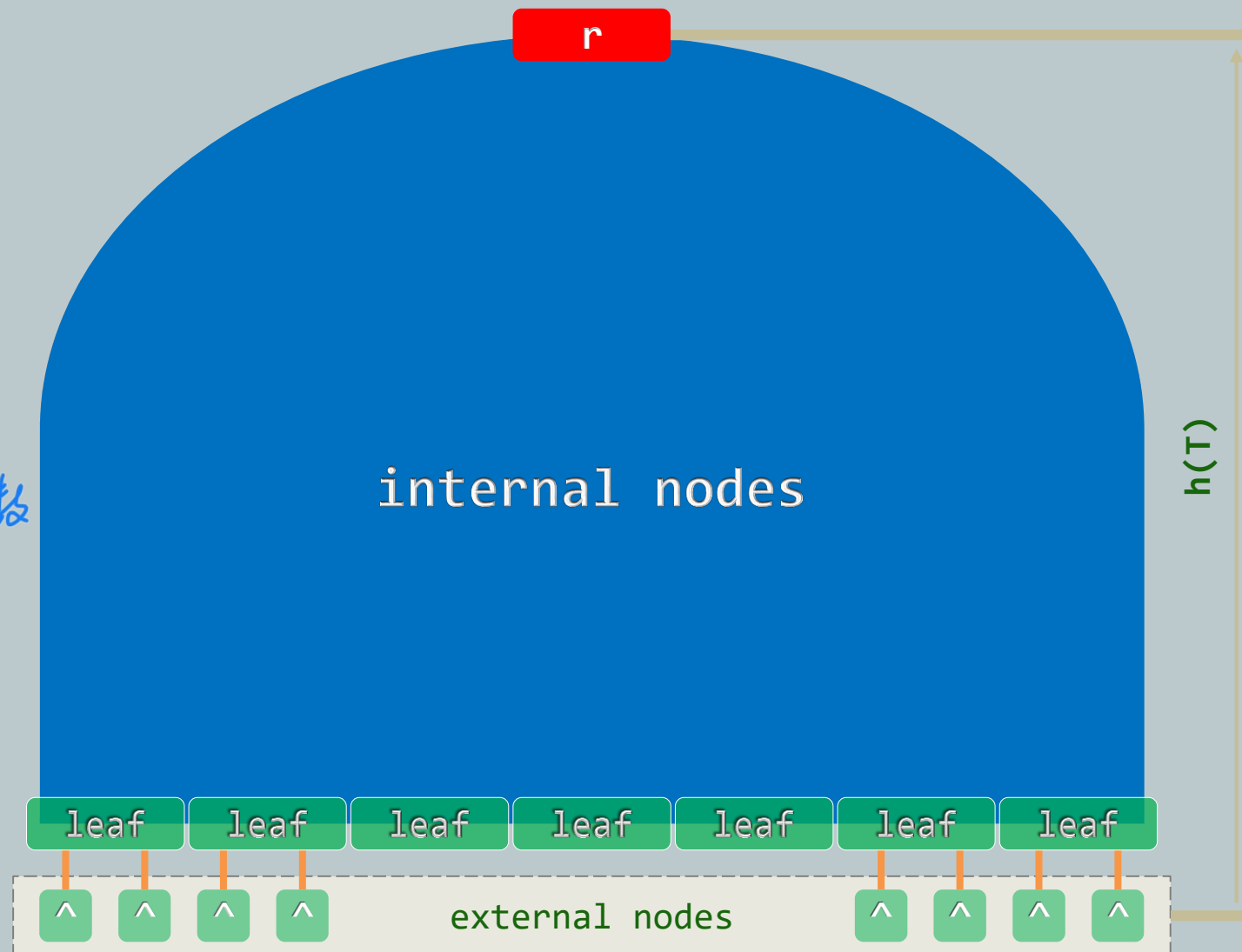
❖ 反过来, 分支数也不能太少

- 树根: $2 \leq n+1$ 树根处的分支数
- 其余: $\lceil m/2 \rceil \leq n+1$

❖ 故亦称作 $(\lceil m/2 \rceil, m)$ -树

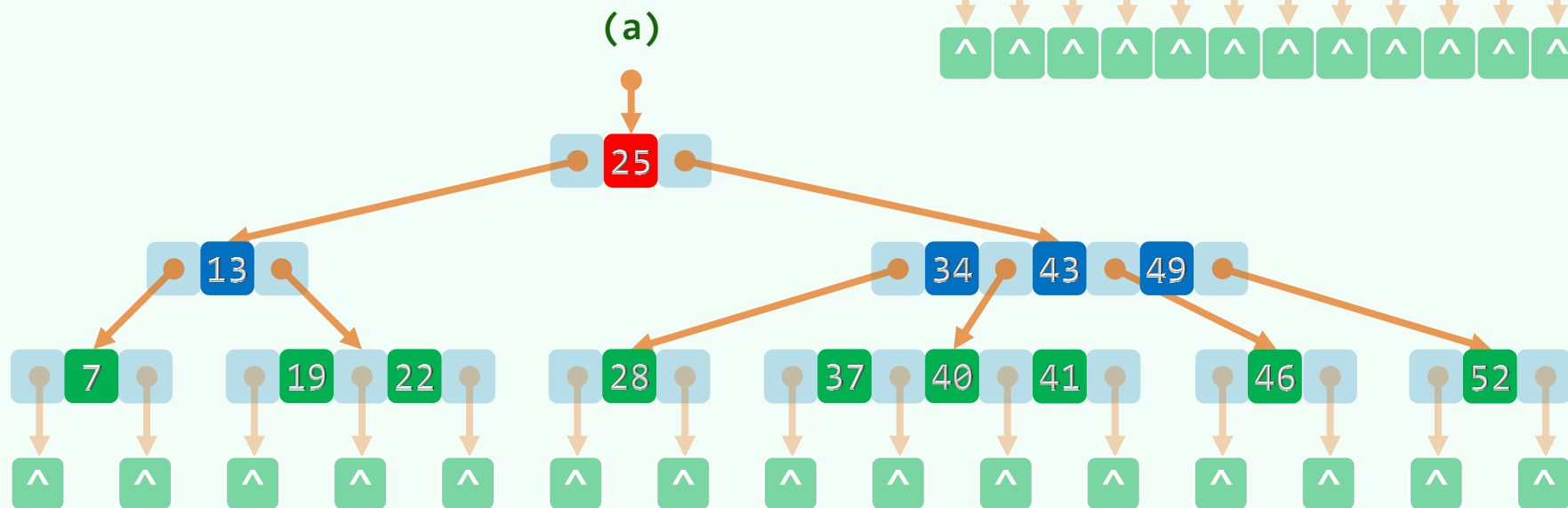
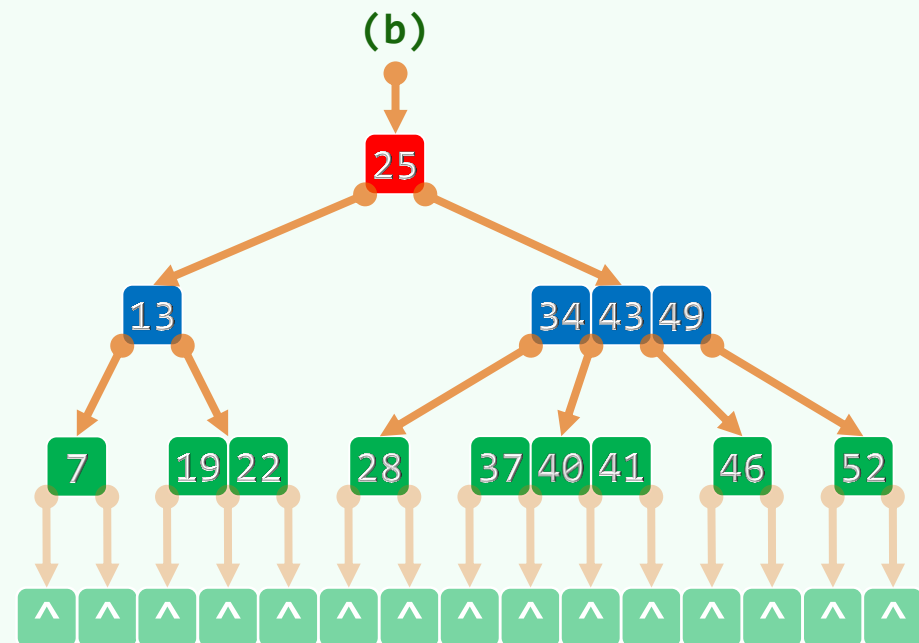
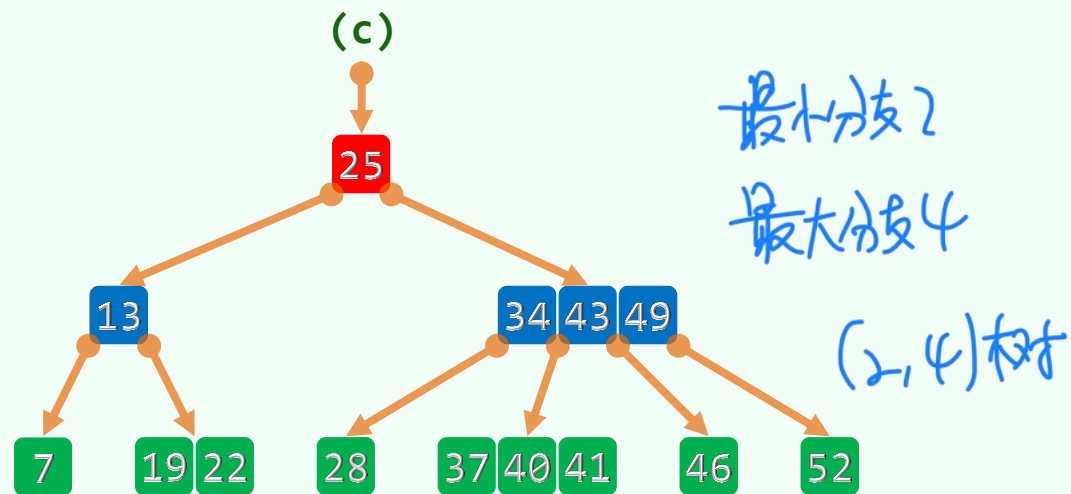
- (3,5)-树
- (9,18)-树
- ...

最大分支数



紧凑表示

这三棵树都是一样的树



实例

❖ $m = 3$: 2-3-树, $(2,3)$ -树

$$\left(\left\lceil \frac{m}{2} \right\rceil, m\right)$$

最简单的B-树 // J. Hopcroft, 1970

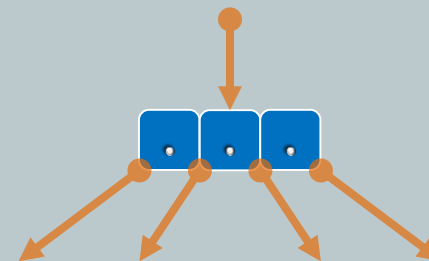
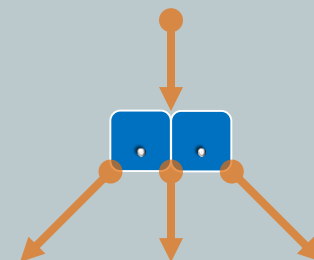
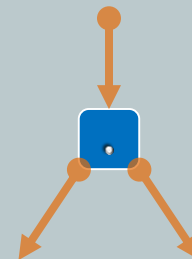
- 各（内部）节点的分支数，可能是2或3
- 各节点所含key的数目，可能是1或2

$$\text{key} \leq m-1$$

❖ $m = 4$: 2-3-4-树, $(2,4)$ -树

- 各节点的分支数，可能是2、3或4
- 各节点所含key的数目，可能是1、2或3

❖ 留意把玩4阶B-树，稍后对于理解红黑树大有裨益



BTNode

```
template <typename T> struct BTNode { //B-树节点
```

```
    BTNodePosi<T> parent; //父
```

```
    Vector<T> key; //关键码 (总比孩子少一个)
```

```
    Vector< BTNodePosi<T> > child; //孩子
```

```
    BTNode() { parent = NULL; child.insert( NULL ); }
```

```
    BTNode( T e, BTNodePosi<T> lc = NULL, BTNodePosi<T> rc = NULL ) {
```

```
        parent = NULL; //作为根节点
```

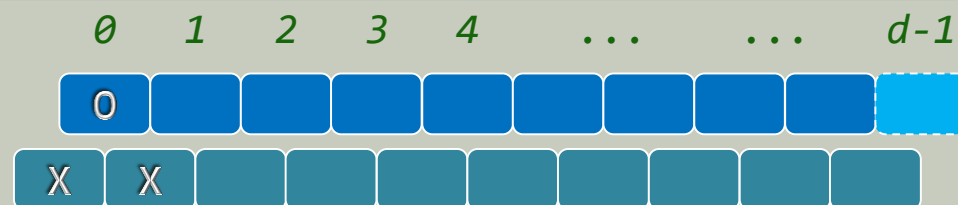
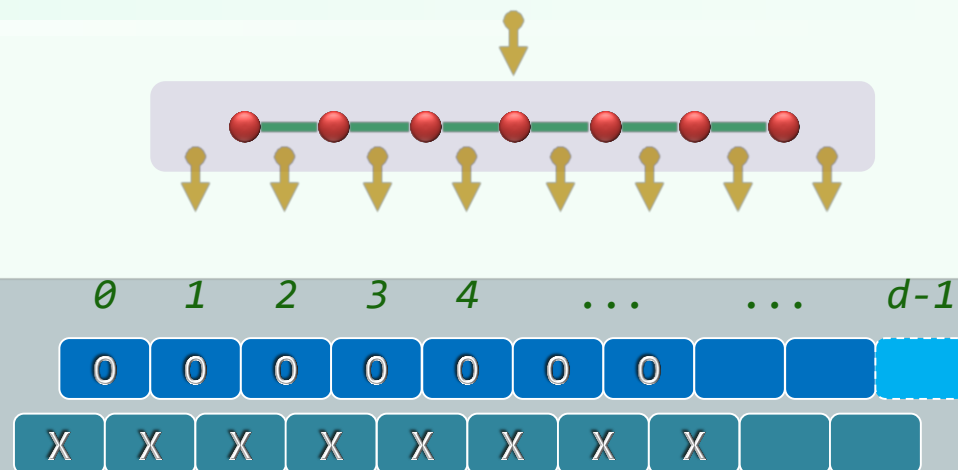
```
        key.insert( e ); //仅一个关键码, 以及
```

```
        child.insert( lc ); if ( lc ) lc->parent = this; //左孩子
```

```
        child.insert( rc ); if ( rc ) rc->parent = this; //右孩子
```

```
    }
```

```
};
```



BTree

```
template <typename T> using BTNodePosi = BTNode<T>*; //B-树节点位置

template <typename T> class BTree { //B-树
protected:
    int _size; int _m; BTNodePosi<T> _root; //关键码总数、阶次、根
    BTNodePosi<T> _hot; //search()最后访问的非空节点位置
    void solveOverflow( BTNodePosi<T> ); //因插入而上溢后的分裂处理
    void solveUnderflow( BTNodePosi<T> ); //因删除而下溢后的合并处理
public:
    BTNodePosi<T> search( const T & e ); //查找
    bool insert( const T & e ); //插入
    bool remove( const T & e ); //删除
};
```

高级搜索树

B-树：查找

算法

从（常驻RAM的）根节点开始

只要当前节点不是外部节点

在当前节点中**顺序查找** // RAM内部

若找到目标关键码，则

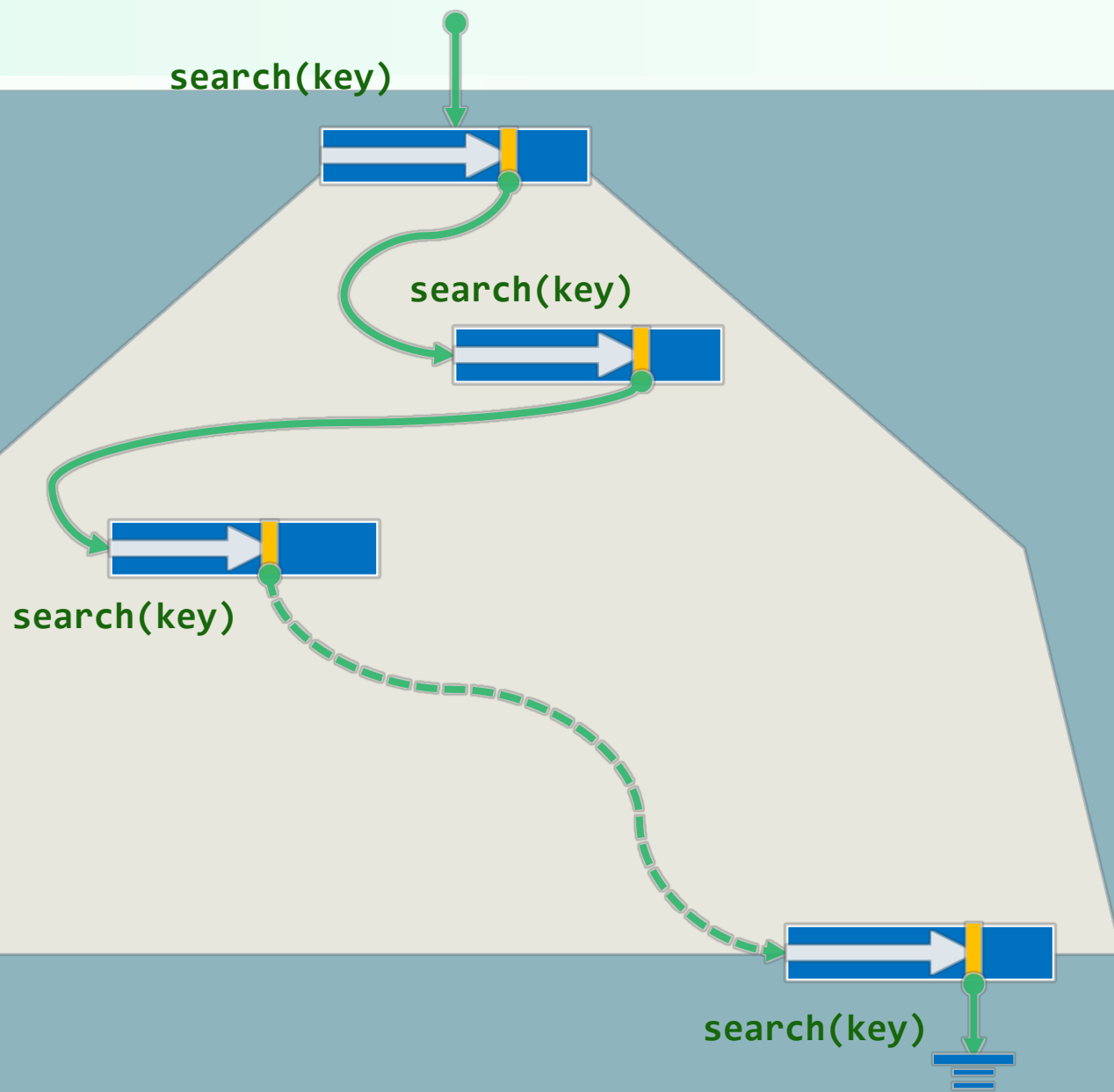
返回**查找成功**

否则 // 止于某一向下的引用

沿引用找到孩子节点

将其**读入内存** // I/O耗时

返回**查找失败**

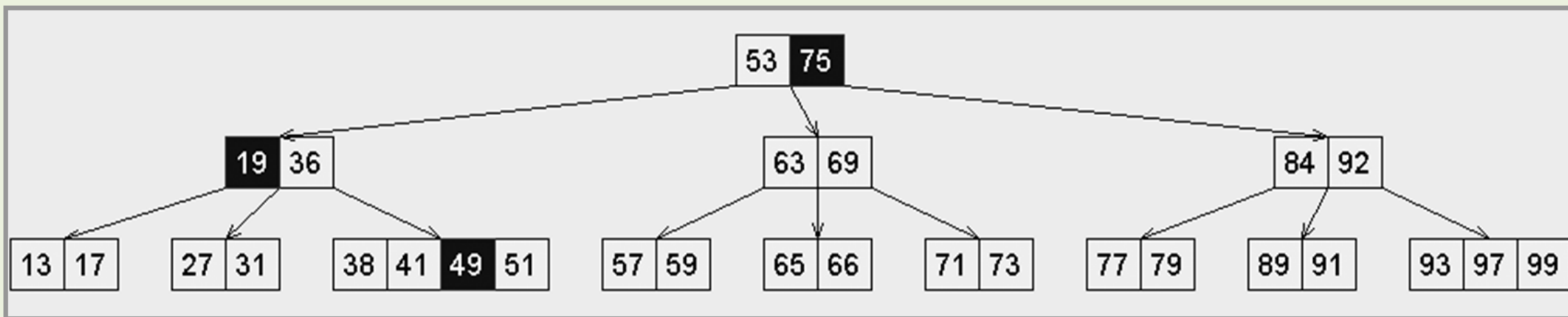


实例

❖ (3,5)-树: 53 97 36 89 41 75 19 84 77 79 51 57 99 91
 92 93 17 73 13 66 59 49 63 65 71 69 27 31 38

成功查找: 75, 19, 49

失败查找: 5, 45



实现

```
❖ template <typename T> BNodePosi<T> BTree<T>::search( const T & e ) {
```

```
    BNodePosi<T> v = _root; _hot = NULL; //从根节点出发
```

```
    while ( v ) { //逐层深入地
```

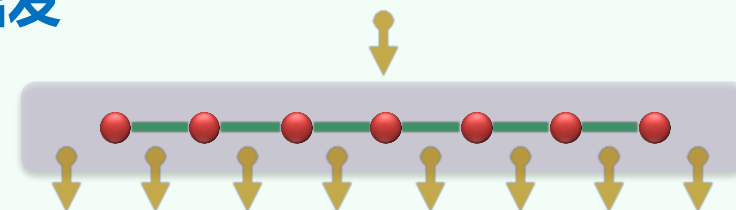
```
        Rank r = v->key.search( e ); //在当前节点对应的向量中顺序查找
```

```
        if ( 0 <= r && e == v->key[r] ) return v; //若成功，则返回；否则...
```

```
        _hot = v; v = v->child[ r + 1 ]; //沿引用转至对应的下层子树，并载入其根 (I/O)
```

```
    } //若因!v而退出，则意味着抵达外部节点
```

```
    return NULL; //失败
```



错位
关键字从0开始
关键字从1开始

0	1	2	3	4	...	...	d-1
0	0	0	0	0	0	0	
X	X	X	X	X	X	X	

```
}
```

性能

❖ 约定：根节点常驻RAM

❖ 忽略内存中的查找

运行时间主要取决于I/O次数

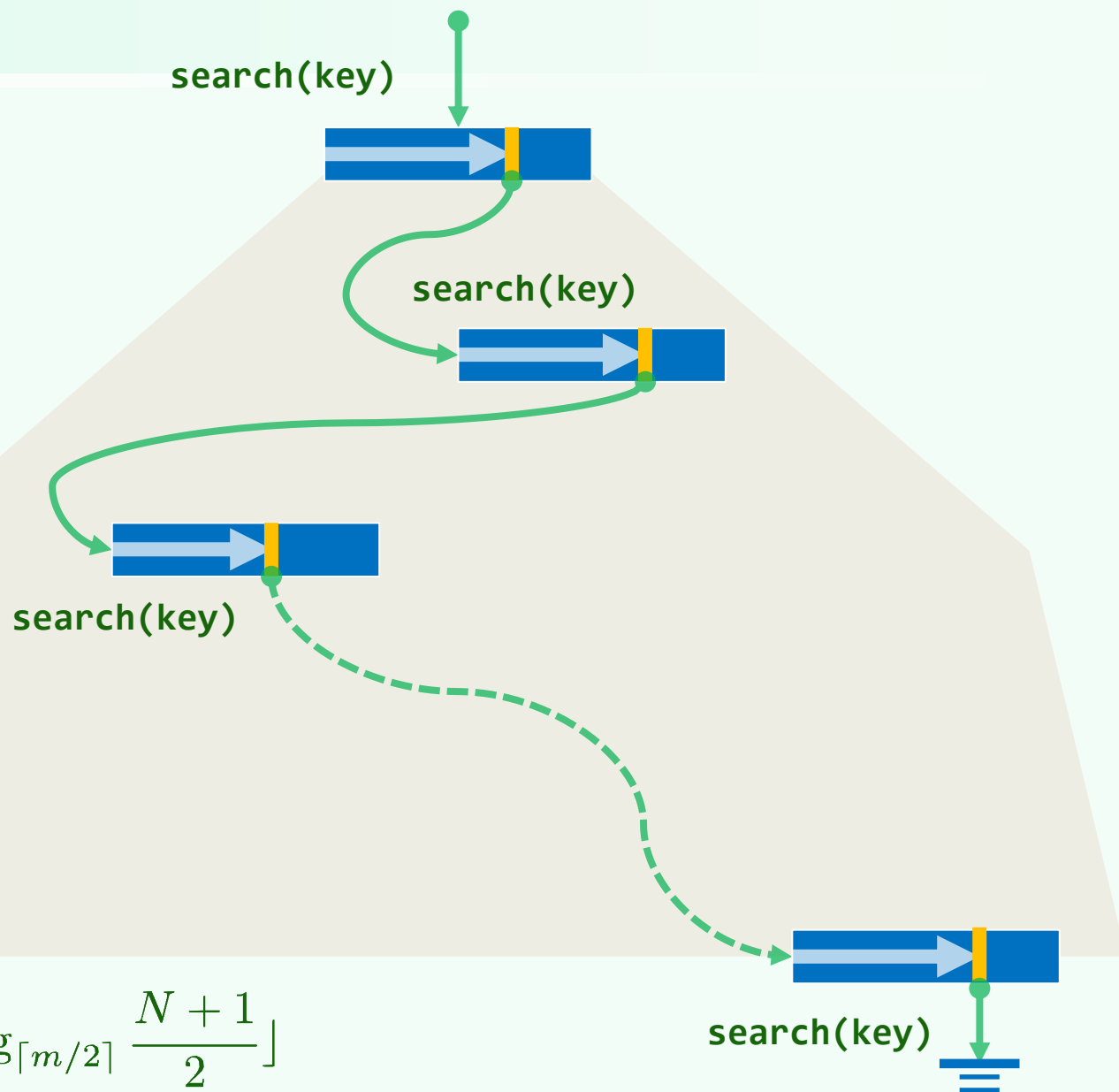
而不是内存查找

❖ 在每一深度至多一次I/O

❖ 故运行时间 = $\mathcal{O}(\log n)$

主要取决于树高

❖ 可以证明： $\log_m (N + 1) \leq h \leq 1 + \left\lceil \log_{\lceil m/2 \rceil} \frac{N + 1}{2} \right\rceil$



最大树高

❖ 含N个关键码的m阶B-树，可能有多“高”？

❖ 为此，内部节点应尽可能地“瘦”

$$n_k \geq 2 \times \lceil m/2 \rceil^{k-1}, \quad \forall k > 0$$

❖ 考查外部节点所在的那层： → 最小分支数

外部节点数 $N+1 = n_h \geq \underline{2} \times \lceil m/2 \rceil^{h-1}$ 反证法

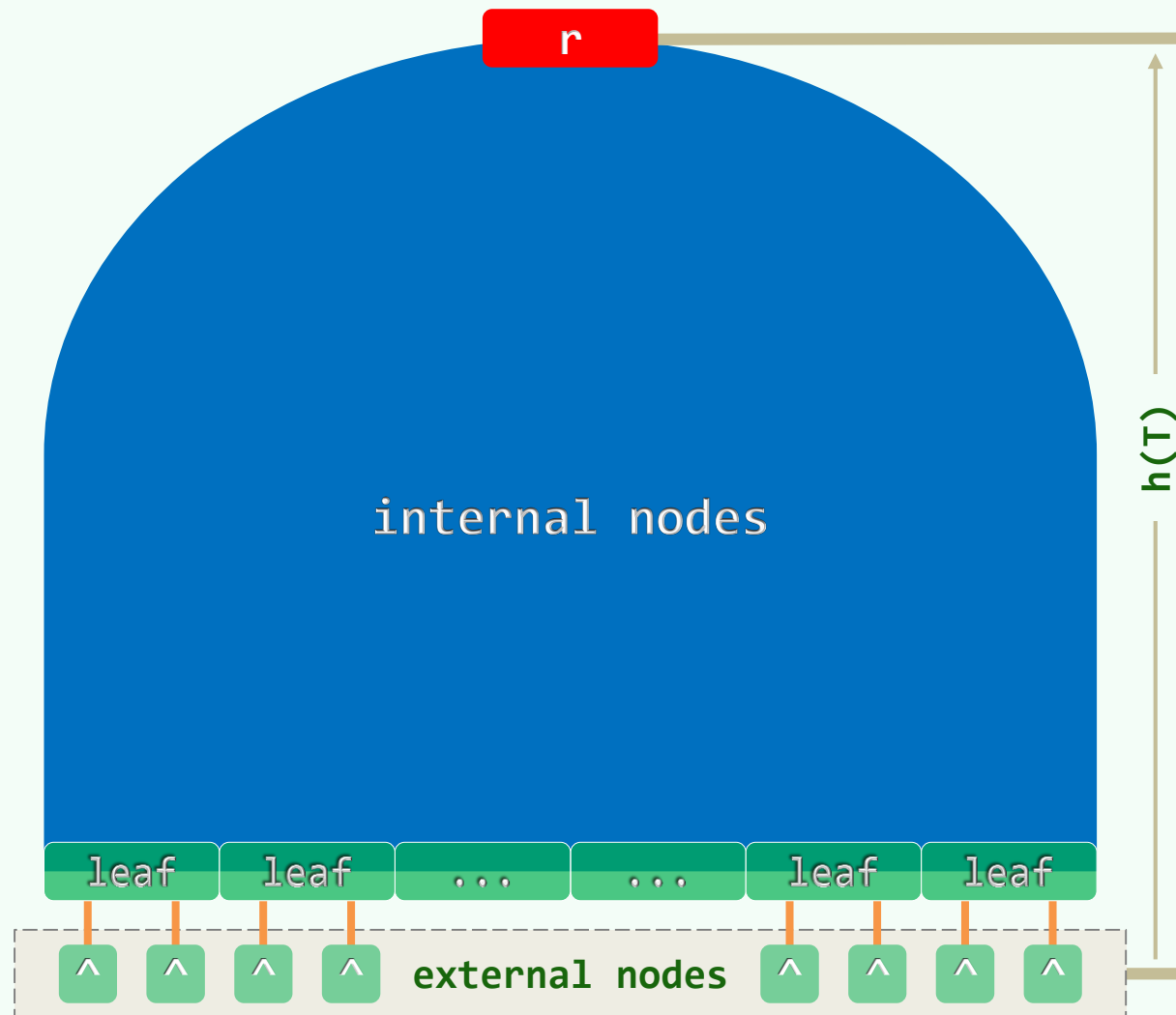
$$h \leq 1 + \left\lfloor \log_{\lceil \frac{m}{2} \rceil} \frac{N+1}{2} \right\rfloor = \mathcal{O}(\log_m N)$$

❖ 相对于BBST:

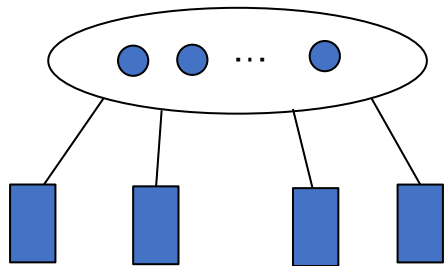
$$\log_{\lceil \frac{m}{2} \rceil} (N/2) / \log_2 N = 1/(\log_2 m - 1)$$

若取 $m = 256$ ，树高约降低至 **1/7**

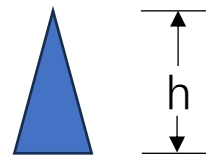
...用**4**年上完大学，还是**28**年？



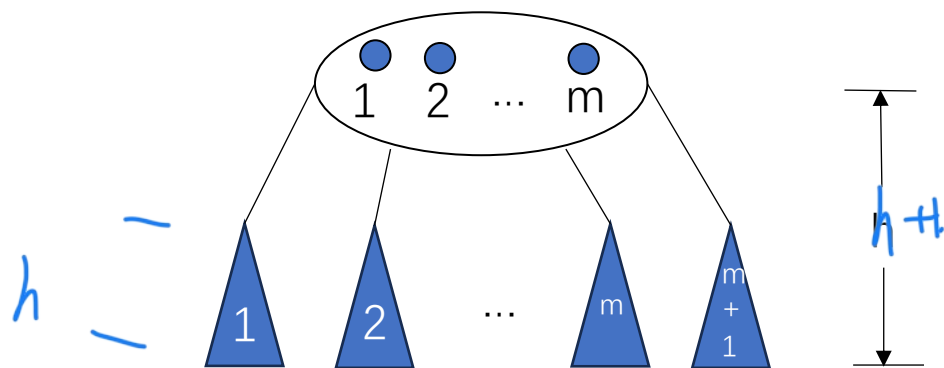
若 m 阶B-树中具有 N 个关键字，则叶子节点即查找不成功的结点为 $N+1$ 个。



最下一层成立，
叶子节点比关键字多1
外部



假设高为 h 的子树都成立，即叶子节点比关键字多1。



- 对于高为 $h+1$ 的子树，假设根的关键字有 m 个，则有 $m+1$ 个子树。
- 每个子树的叶子节点都比对应的关键字多1，那么所有子树的叶子节点比关键字多 $m+1$ 。
- 那么加上根节点的 m 个关键字，所有叶子节点就比所有关键字多1。

$$m+1-m=1$$

每个子树的外部节点都多1，有 $m+1$ 棵树，多 $m+1$

最小树高

❖ 含 N 个关键码的 m 阶B-树，可能有多“矮”？

❖ 为此，内部节点应尽可能“胖”

$$n_k \leq m^k, \quad \forall k \geq 0$$

❖ 依然，考查外部节点所在的那层

$$N + 1 = n_h \leq m^h \quad \text{最大分支 } m^h$$

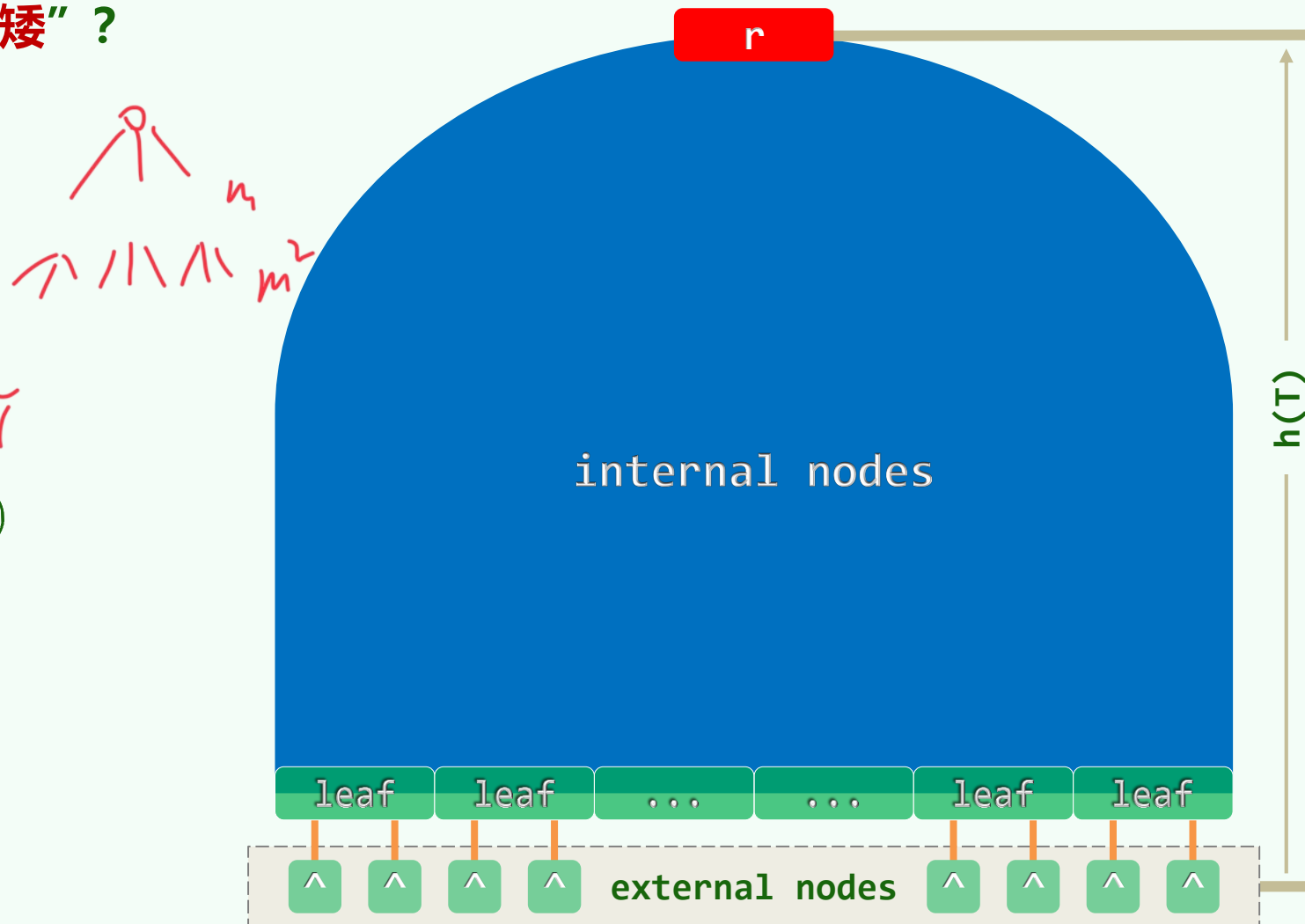
$$h \geq \lceil \log_m (N + 1) \rceil = \Omega(\log_m N)$$

❖ 相对于BBST:

$$(\log_m N - 1) / \log_2 N$$

$$= \log_m 2 - \log_N 2 \approx 1 / \log_2 m$$

若取 $m = 256$ ，树高约降低至**1/8**



高级搜索树

B-树： 插入

算法

```
template <typename T> bool BTree<T>::insert( const T & e ) {
```

```
    BTreeNodePosi<T> v = search( e );
```

记录 hot

```
    if ( v ) return false; //确认e不存在
```

```
    Rank r = _hot->key.search( e ); //在节点_hot中确定插入位置
```

≤ 当前最大元素

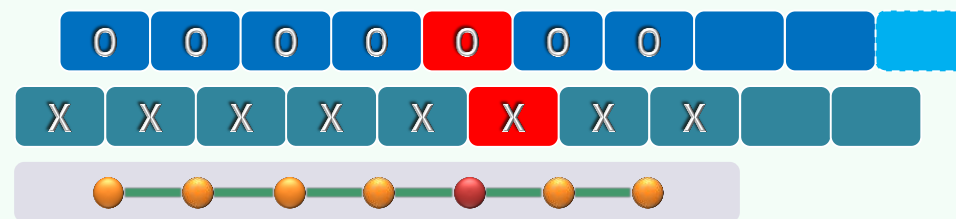
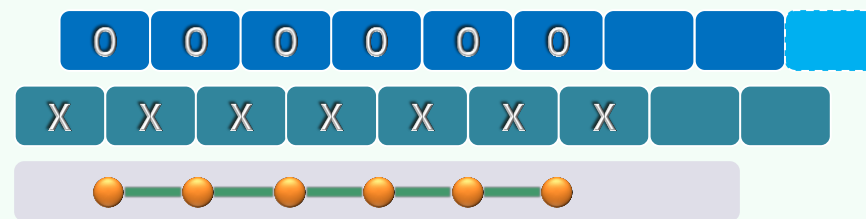
```
    _hot->key.insert( r+1, e ); //将新关键码插至对应的位置
```

```
    _hot->child.insert( r+2, NULL ); _size++; //创建一个空子树指针
```

```
    solveOverflow( _hot ); //若上溢，则分裂
```

```
    return true; //插入成功
```

```
}
```



分裂

❖ 设上溢节点中的关键码依次为:

$$\{ k_0, k_1, \dots, k_{m-1} \}$$

❖ 取中位数 $s = \lfloor m/2 \rfloor$, 以关键码 k_s 为界划分为

$$\{ k_0, \dots, k_{s-1} \} \quad \{ k_s \} \quad \{ k_{s+1}, \dots, k_{m-1} \}$$

❖ 关键码 k_s 上升一层, 并分裂 (split)

以所得的两个节点作为左、右孩子

❖ 不难验证, 如此分裂后

左、右孩子所含**关键码数目**, 依然符合m阶B-树的条件



再分裂

❖ 若上溢节点的父亲**本已饱和**，则在接纳被提升的关键码之后，也将上溢
此时，大可套用前法，继续分裂

❖ 上溢可能持续发生，并逐层**向上传播**

纵然最坏情况，亦不过到根 //若果真抵达树根...

❖ 可令被提升的关键码自成节点，作为新的树根

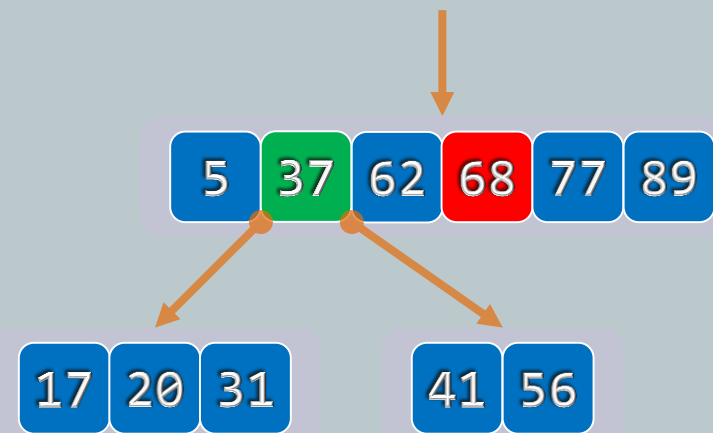
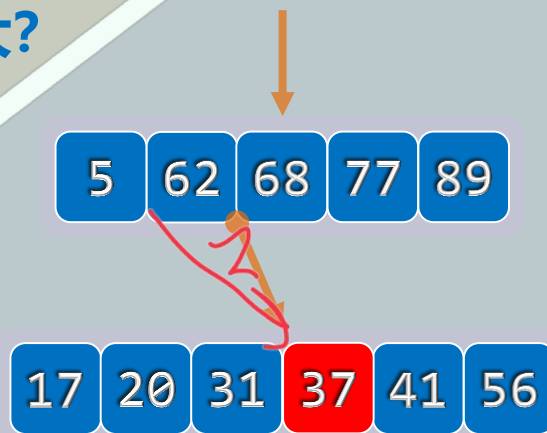
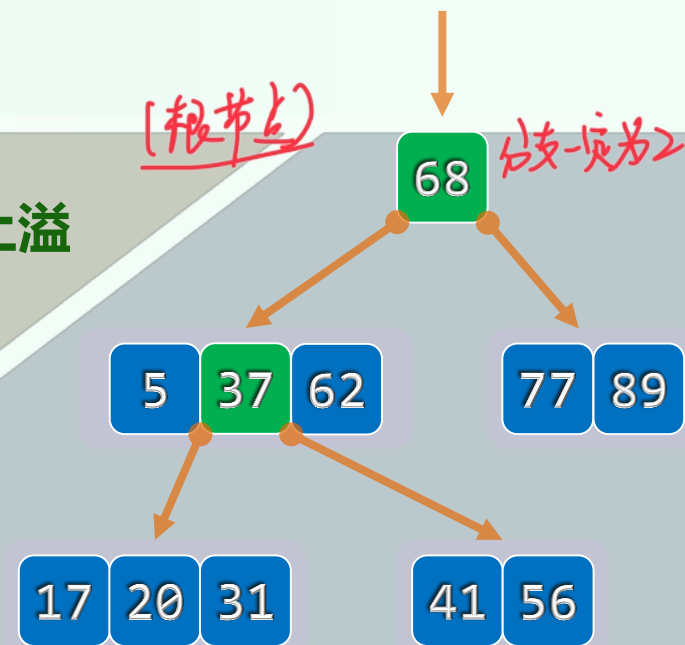
这是B-树增高的**唯一可能** //概率多大?

❖ 注意：新生的树根仅有**两个分支**

❖ 总体执行时间正比于

分裂次数, $O(h)$ *最多传递到根*

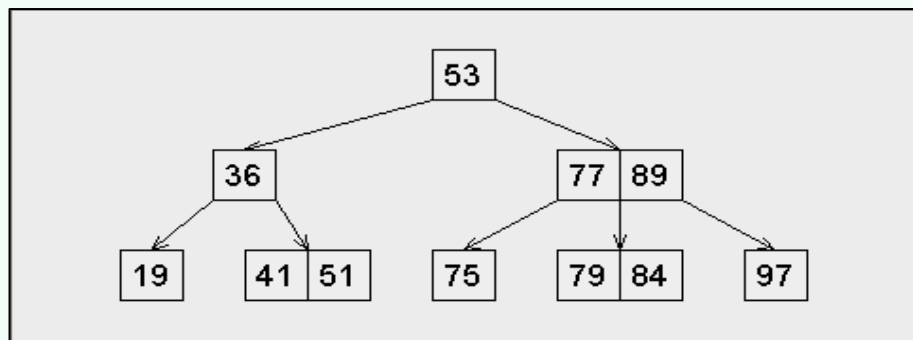
$\log_m N$



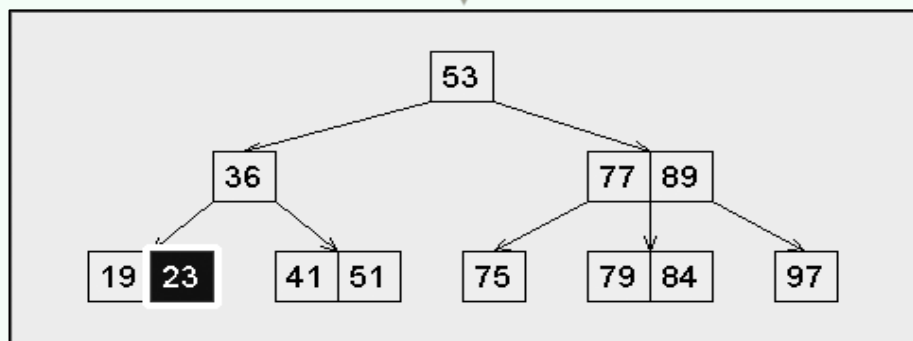
实例: (2,3)-树

$(\lceil \frac{m}{2} \rceil, m)$

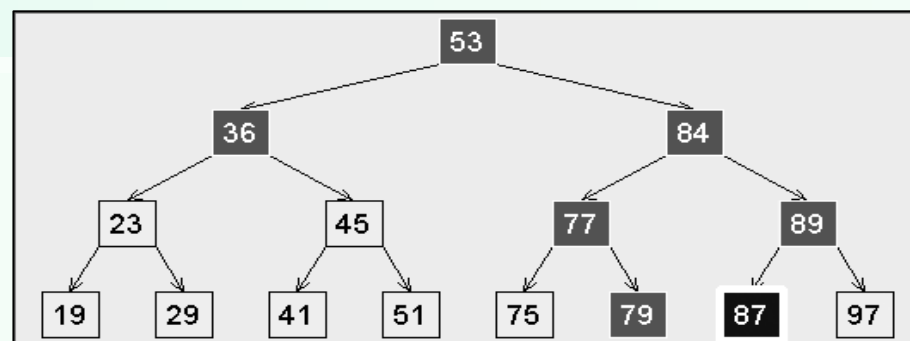
关键的(1,2)



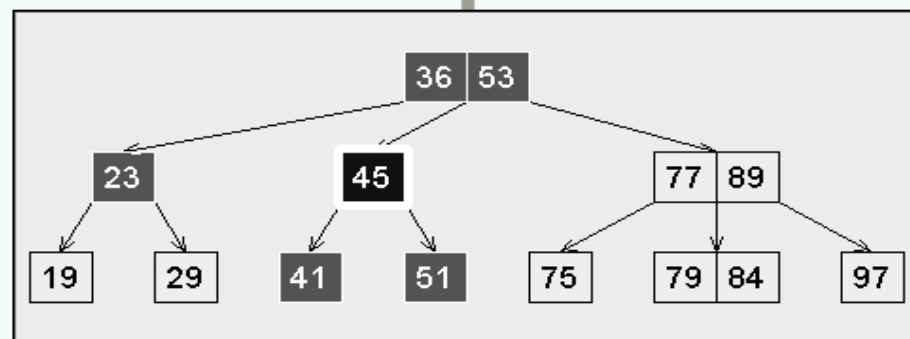
insert(23) //无需分裂



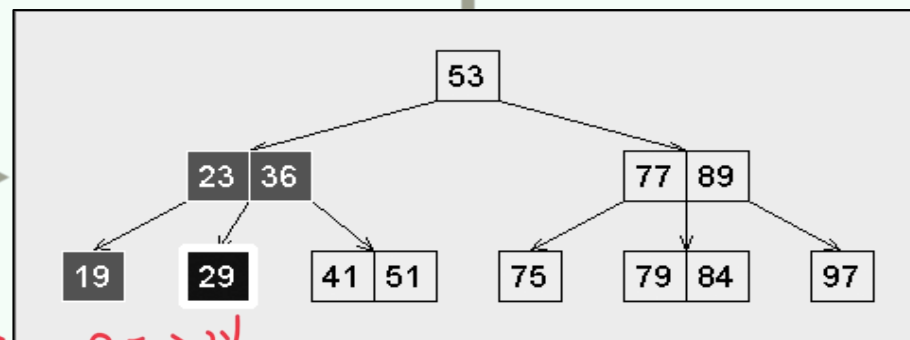
insert(29) //分裂一次



insert(87) //分裂到根



insert(45) //分裂两次



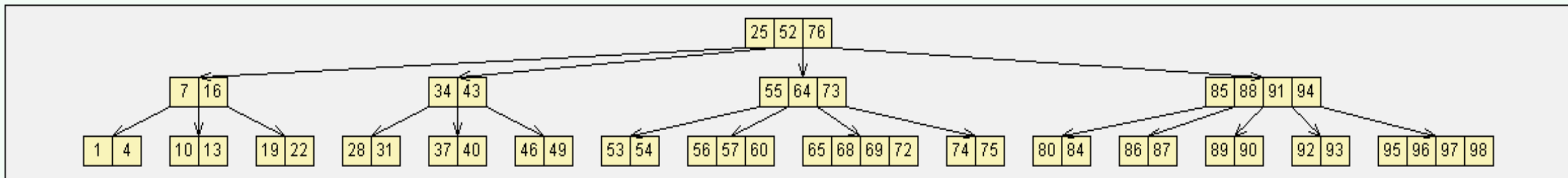
❖ 53 97 36 89 41 75 19 84 77 79 51

19 23 29

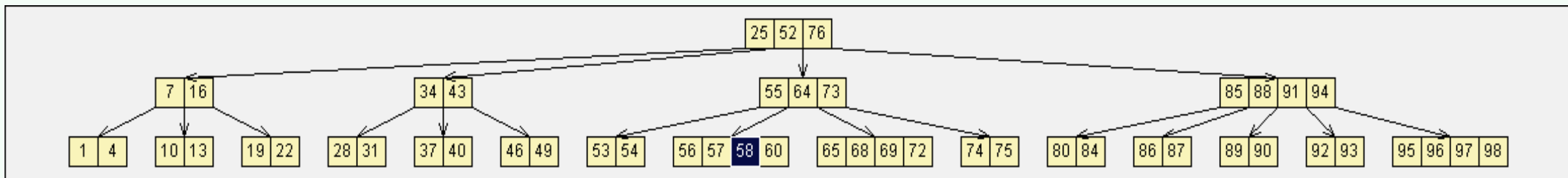
放29到分裂

实例：(3,5)-树

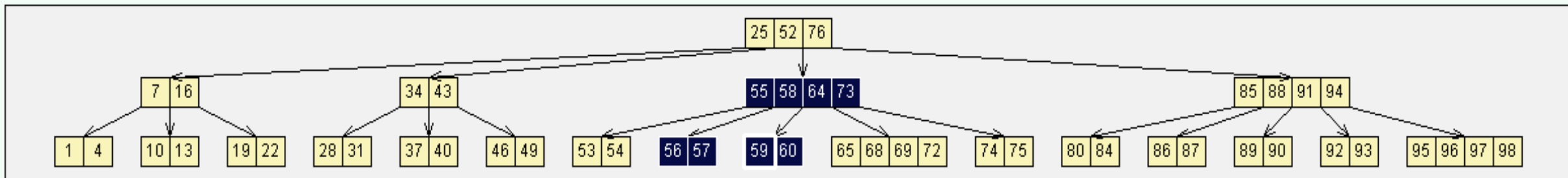
1 4 7 10 13 16 19 22 25 28 31 34 37 40 43 46 49 52 56 60 64 68 72 76 80 84 53 54 55 85 86 87 88 89 90 91 92 93 94 95 96 97 98 73 74 75 57 65 69



insert(58) //无需分裂

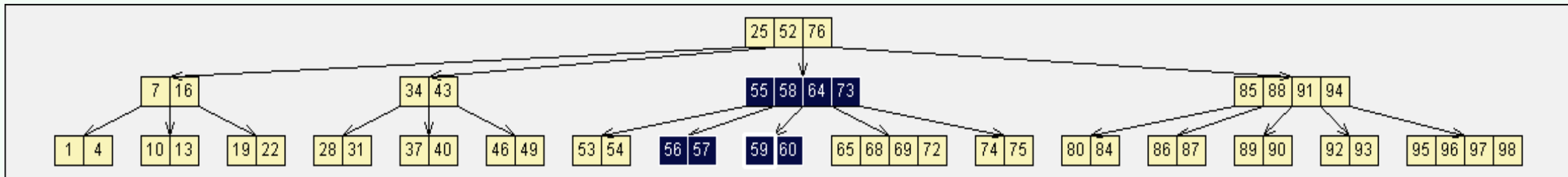


insert(59) //分裂 1 次

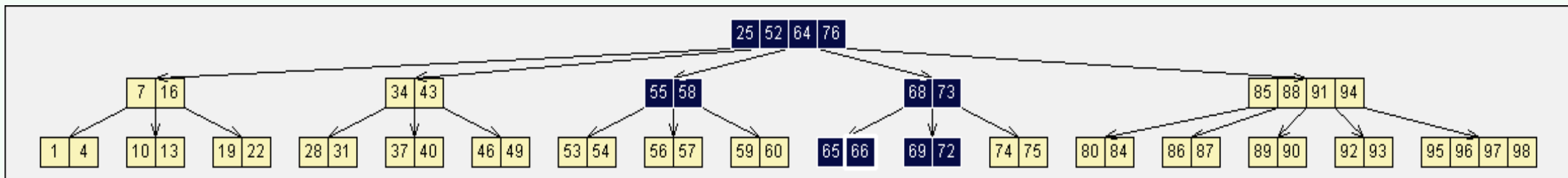


实例：(3,5)-树

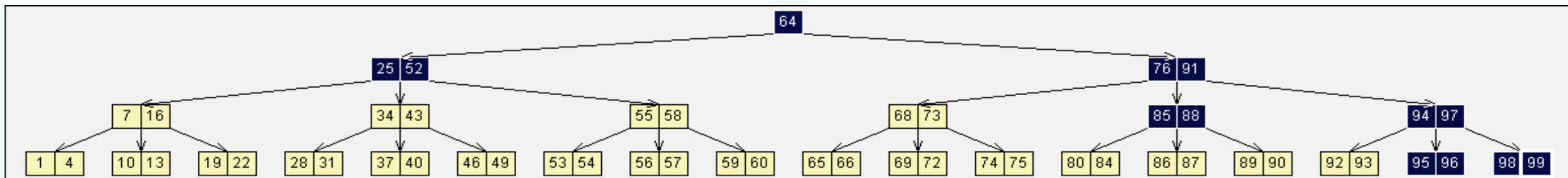
insert(59) //分裂 1 次



insert(66) //分裂 2 次



insert(99) //分裂到根



上溢修复 (1/2)

```
template <typename T> void BTree<T>::solveOverflow( BTreeNodePosi<T> v ) {
```

```
    if ( _m > v->key.size() ) return; //递归基: 不再上溢
```

```
    Rank s = _m / 2; //轴点 (此时_m = key.size() = child.size() - 1)
```

```
    BTreeNodePosi<T> u = new BTreeNode<T>(); //注意: 新节点已有一个空孩子
```

new出来有个vector

```
    for ( Rank j = 0; j < _m - s - 1; j++ ) { //分裂出右侧节点u (效率低可改进)
```



```
        u->child.insert( j, v->child.remove( s + 1 ) ); //v右侧_m-s-1个孩子
```

```
        u->key.insert( j, v->key.remove( s + 1 ) ); //v右侧_m-s-1个关键码
```

```
    }
```

左孩子比关键码多一个

```
    u->child[ _m - s - 1 ] = v->child.remove( s + 1 ); //移动v最靠右的孩子
```

```
/* TBC */
```

上溢修复 (2/2)

```
if ( u->child[ 0 ] ) //若u的孩子们非空，则统一令其以u为父节点
```

```
    for ( Rank j = 0; j < _m - s; j++ ) u->child[ j ]->parent = u;
```

```
BTNodePosi<T> p = v->parent; //v当前的父节点p
```

```
if ( ! p ) //若p为空，则创建之（全树长高一层，新根节点恰好两度）
```

```
    { _root = p = new BTNode<T>(); p->child[0] = v; v->parent = p; }
```

```
Rank r = 1 + p->key.search( v->key[0] ); //p中指向u的指针的秩
```

```
p->key.insert( r, v->key.remove( s ) ); //轴点关键码上升
```

```
p->child.insert( r + 1, u ); u->parent = p; //新节点u与父节点p互联
```

```
solveOverflow( p ); //上升一层，如有必要则继续分裂——至多（尾）递归 $O(\log n)$ 层
```

```
}
```

高级搜索树

B-树： 删除

算法：确保目标在叶子中

```
template <typename T>

bool BTree<T>::remove( const T & e ) {

    BTreeNodePosi<T> v = search( e );

    if ( ! v ) return false; //确认e存在

    Rank r = v->key.search(e); //e在v中的秩

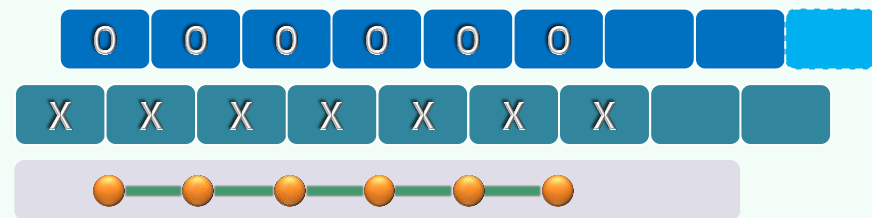
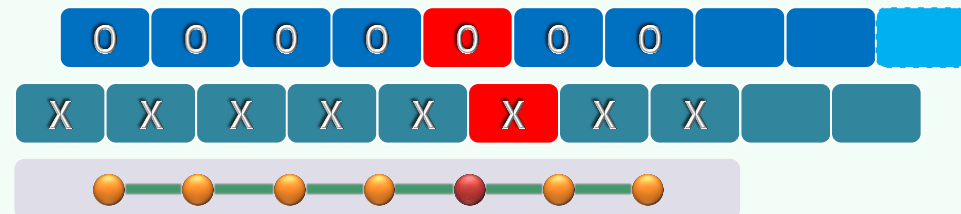
    if ( v->child[0] ) { /* 若v非叶子，则可经过腾挪，确保... */ }

    //assert: 至此，v必位于最底层，且其中第r个关键字就是待删除者

    v->key.remove( r ); v->child.remove( r + 1 ); _size--;

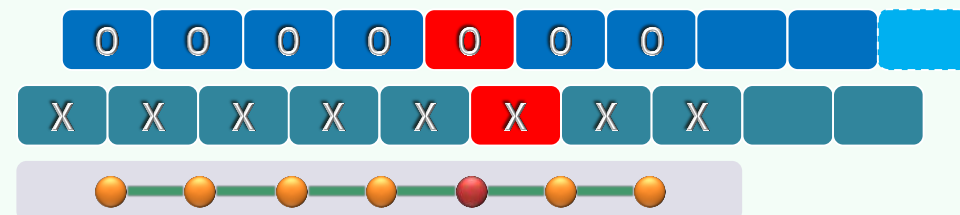
    solveUnderflow( v ); return true; //如有必要，需做旋转或合并

}
```



算法：腾挪 = 与后继交换

```
template <typename T>
bool BTree<T>::remove( const T & e ) {
    /* ..... */
```



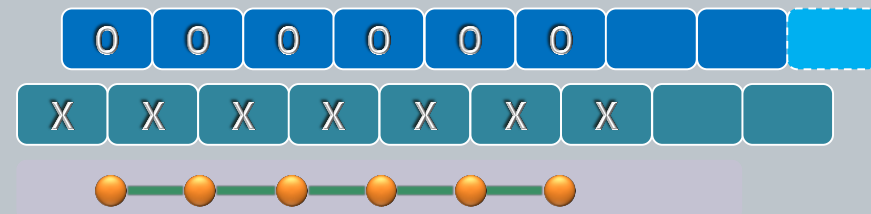
```
if ( v->child[0] ) { //若v非叶子，则
```

```
    BTNodePosi<T> u = v->child[r + 1]; //在右子树中
```

```
    while ( u->child[0] ) u = u->child[0]; //一直向左，即可找到e的后继（必在底层）
```

```
    v->key[r] = u->key[0]; v = u; r = 0; //交换
```

```
}
```



```
//assert: 至此，v必位于最底层，且其中第r个关键码就是待删除者
```

```
/* ..... */
```

```
}
```

旋转

❖ 非根节点[V]下溢时，必恰有 $\lceil m/2 \rceil - 2$ 个关键码和 $\lceil m/2 \rceil - 1$ 个分支

❖ 视其左、右兄弟[L]、[R]的规模，可分三种情况加以处理

1) 若[L]存在，且至少包含 $\lceil m/2 \rceil$ 个关键码

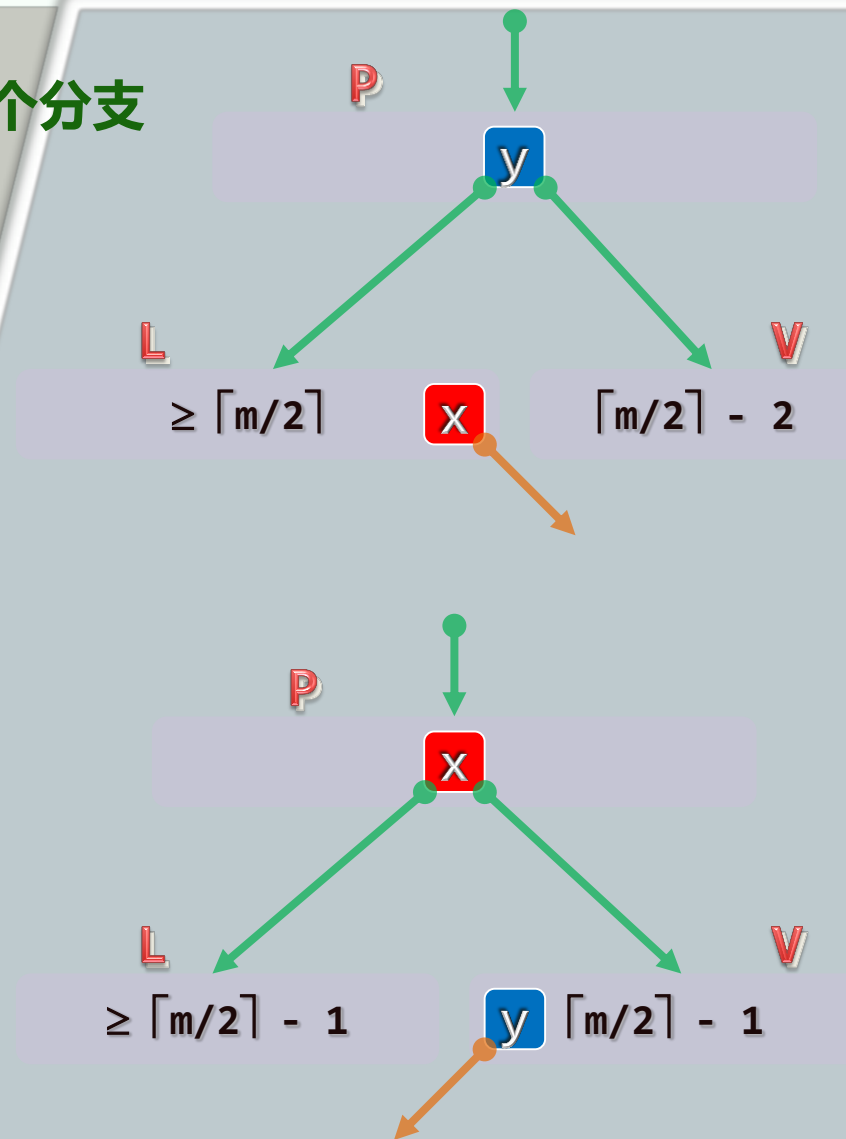
- 将[P]中的分界关键码[y]移至[V]中（作为最小关键码）
- 将[L]中的最大关键码[x]移至[P]中（取代原关键码[y]）

❖ 如此旋转之后，局部乃至全树都重新满足B-树条件

下溢修复完毕

2) 若[R]存在，且至少包含 $\lceil m/2 \rceil$ 个关键码

- 也可旋转，完全对称



合并

3) L 和 R 或不存在，或均不足 $\lceil m/2 \rceil$ 个关键字——即便如此

- L 和 R 仍必有其一（不妨以 L 为例），且
- 恰含 $\lceil m/2 \rceil - 1$ 个关键字

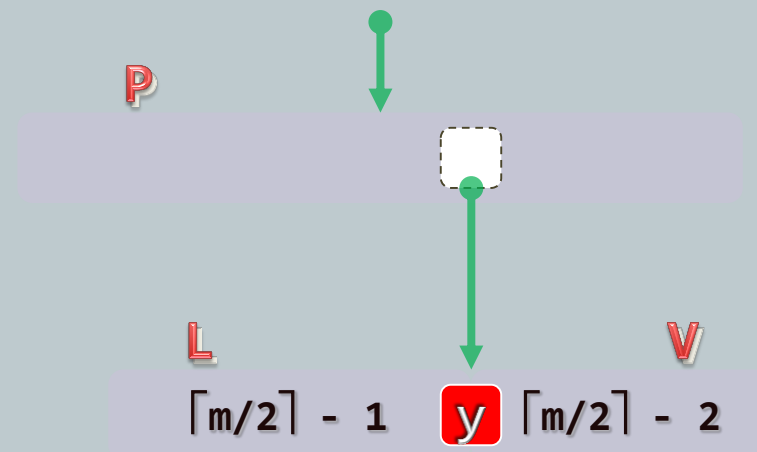
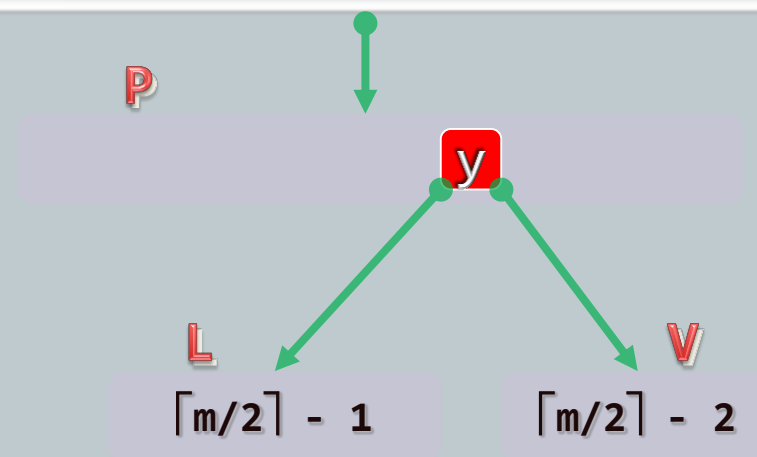
❖ 从 P 中抽出介于 L 和 V 之间的分界关键字 y

- 通过 y 做粘接，将 L 和 V 合成一个节点
- 同时合并此前 y 的孩子引用

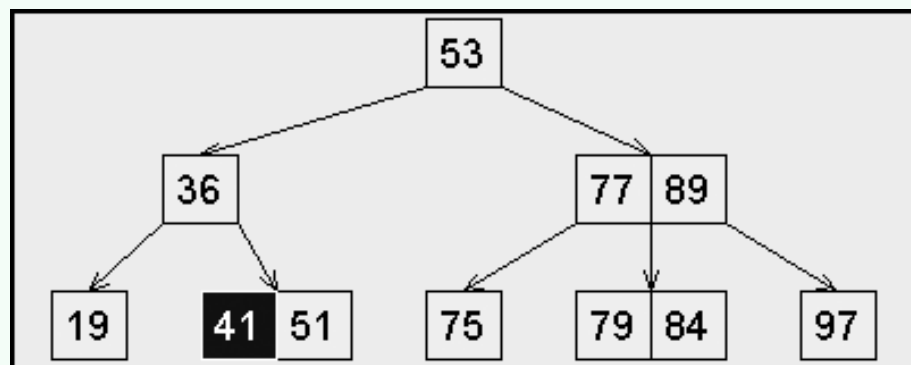
❖ 此处下溢得以修复，但可能继而导致 P 下溢

若果真如此，大可套用前法，继续旋转或合并

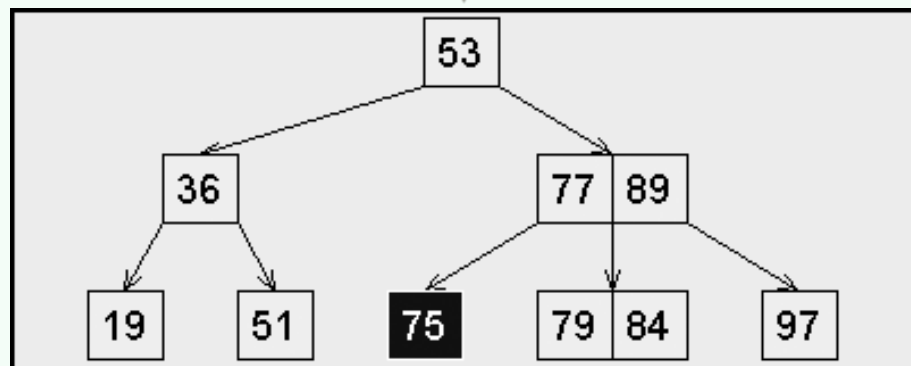
❖ 下溢可能持续发生并向上传播；但至多不过 $O(h)$ 层



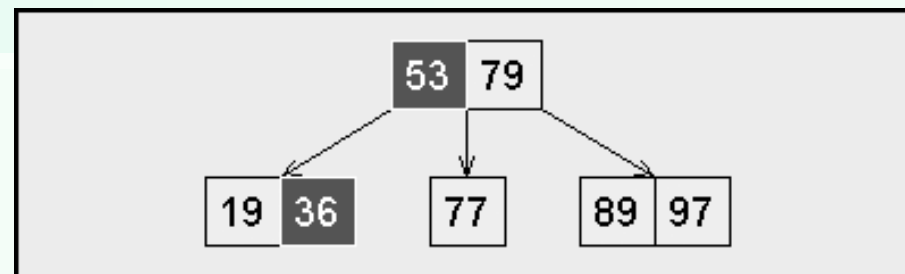
实例：(2,3)-树：底层节点



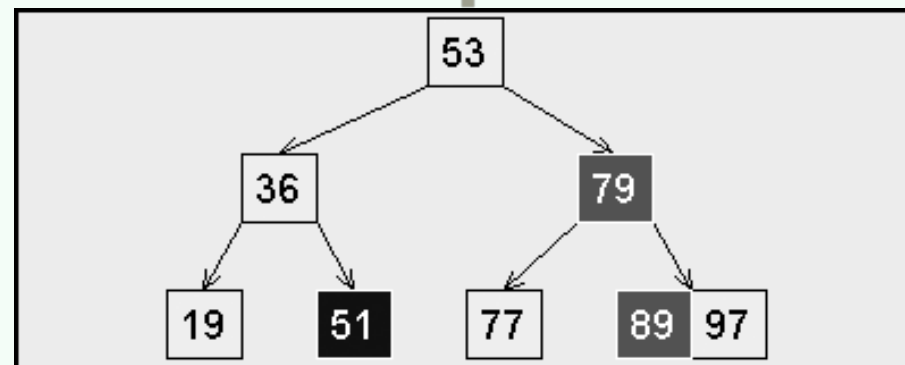
remove(41) //直接删除



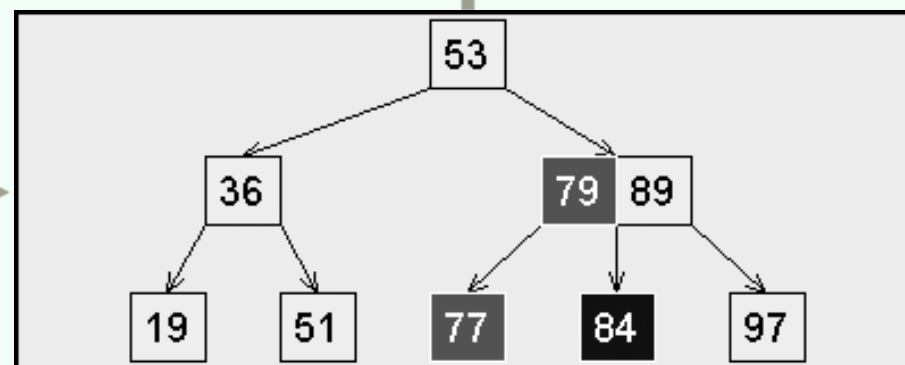
remove(75) //旋转



remove(51) //多次合并



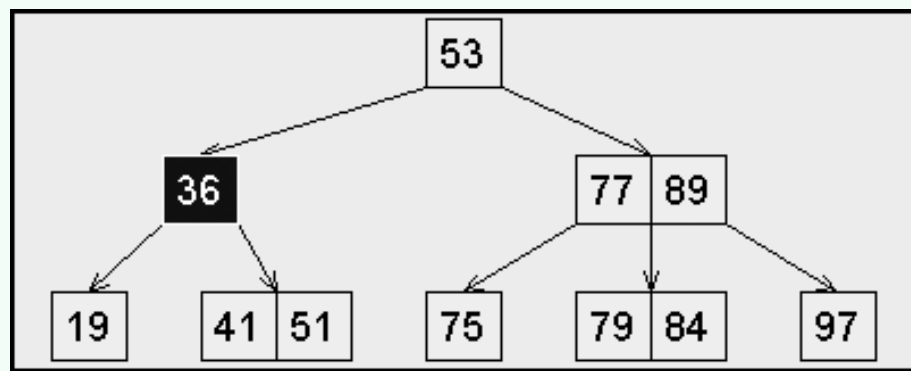
remove(84) //单次合并



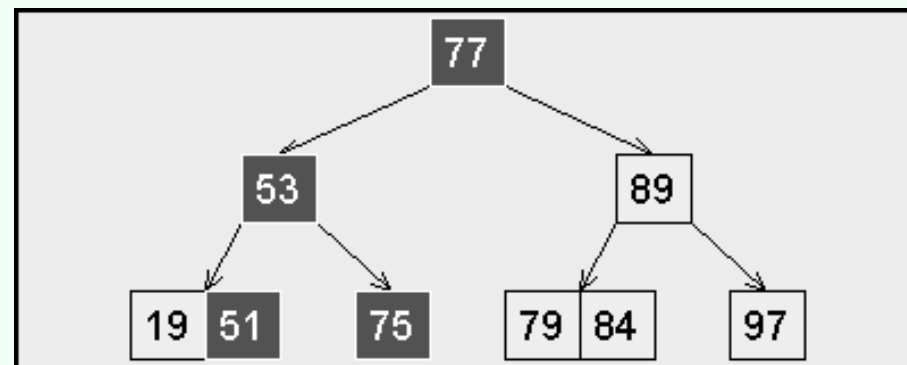
❖ 53 97 36 89 41 75 19 84 77 79 51

实例：(2,3)-树：非底层节点

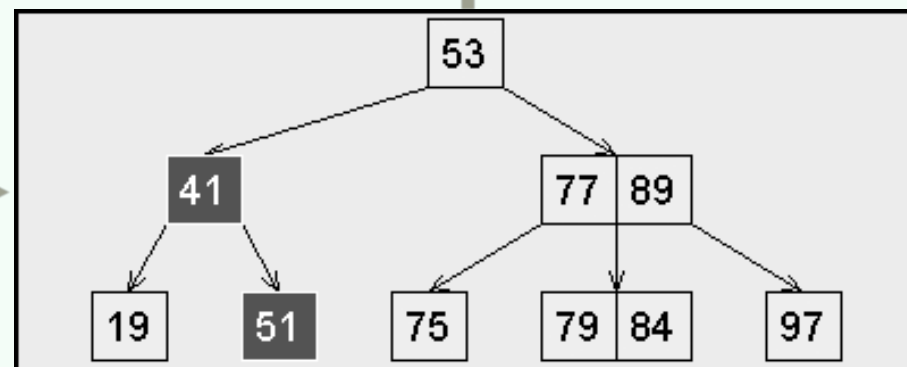
❖ 53 97 36 89 41 75 19 84 77 79 51



remove(36)



remove(41)



下溢修复

```
template <typename T> void BTree<T>::solveUnderflow( BTNodePosi<T> v ) {  
    if ( ( _m + 1 ) / 2 <= v->child.size() ) return; //递归基: v并未下溢  
  
    BTNodePosi<T> p = v->parent; if ( !p ) { /* 递归基: 已到根节点 */ }  
  
    Rank r = 0; while ( p->child[r] != v ) r++; //确定v是p的第r个孩子  
  
    if ( 0 < r ) { /* 情况 #1: 若v的左兄弟存在, 且... */ }  
  
    if ( p->child.size() - 1 > r ) { /* 情况 #2: 若v的右兄弟存在, 且... */ }  
  
    if ( 0 < r ) { /* 与左兄弟合并 */ } else { /* 与右兄弟合并 */ } //情况 #3  
  
    solveUnderflow( p ); //上升一层, 继续分裂——至多递归 $O(\log n)$ 层——典型尾递归  
  
    return;  
}
```

下溢修复：情况#1：旋转（向左兄弟借关键码）

```
if (0 < r) { //若v不是p的第一个孩子，则
    BTNodePosi<T> ls = p->child[r - 1]; //左兄弟必存在
    if ( (_m + 1) / 2 < ls->child.size() ) { //若该兄弟足够“胖”，则
        v->key.insert( 0, p->key[r-1] ); //p借出一个关键码给v（作为最小关键码）
        p->key[r - 1] = ls->key.remove( ls->key.size() - 1 ); //ls的最大关键码转入p
        v->child.insert( 0, ls->child.remove( ls->child.size() - 1 ) );
        //同时ls的最右侧孩子过继给v（作为v的最左侧孩子）
        if ( v->child[0] ) v->child[0]->parent = v;
        return; //至此，通过右旋已完成当前层（以及所有层）的下溢处理
    }
} //情况#2完全对称
```

下溢修复：情况#3：合并 (1/2)

```
if (0 < r) { //与左兄弟合并
```

```
    BTNodePosi<T> ls = p->child[r-1]; //左兄弟必存在
```

```
    ls->key.insert( ls->key.size(), p->key.remove(r - 1) );
```

```
    p->child.remove( r ); //p的第r - 1个关键码转入ls, v不再是p的第r个孩子
```

```
    ls->child.insert( ls->child.size(), v->child.remove( 0 ) );
```

```
    if ( ls->child[ ls->child.size() - 1 ] ) //v的最左侧孩子过继给ls做最右侧孩子
```

```
        ls->child[ ls->child.size() - 1 ]->parent = ls;
```

```
    /* ... TBC ... */
```

```
} else { /* 与右兄弟合并, 完全对称 */ }
```

下溢修复：情况#3：合并 (2/2)

```
if (0 < r) { //与左兄弟合并

    /* ..... */

    while ( !v->key.empty() ) { //v剩余的关键码和孩子，依次转入ls
        ls->key.insert( ls->key.size(), v->key.remove(0) );
        ls->child.insert( ls->child.size(), v->child.remove(0) );
        if ( ls->child[ ls->child.size() - 1 ] )
            ls->child[ ls->child.size() - 1 ]->parent = ls;
    }

    release(v); //释放v

} else { /* 与右兄弟合并，完全对称 */ }
```